Adobe - 100 Technical Interview Questions

Adobe (Software Engineer, New Grad)

With Answers & Diagrams - For BTech Computer Science / IT Students

Object-Oriented Programming

Q1. Explain what a static method is and why it cannot be overridden (only hidden) in Java.

Answer:

A static method belongs to the class itself rather than any instance, and is resolved at compile time based on the reference type, not the runtime object type. Since overriding relies on runtime (dynamic) dispatch, a static method with the same signature in a subclass merely 'hides' the parent's version rather than overriding it - the version called depends on which class name/reference type you use to call it.

Q2. What is operator overloading? Give an example in C++.

Answer:

Operator overloading lets you redefine what an operator like '+' does for a user-defined type. For example, in C++ you could overload '+' for a Complex number class so that c1 + c2 adds their real and imaginary parts, letting Complex objects be used with familiar arithmetic syntax.

Q3. Explain the purpose of the 'this' and 'super' keywords in Java.

Answer:

'this' refers to the current object instance, used to distinguish instance fields from parameters with the same name or to pass the current object elsewhere. 'super' refers to the immediate parent class, used to call a parent's constructor, method, or access a parent field that's been hidden by the subclass.

Q4. What is the difference between composition and inheritance? Which is generally preferred and why?

Answer:

Composition means a class holds a reference to another class as a field ('has-a', e.g. a Car has an Engine), while inheritance means a class extends another ('is-a', e.g. a Car is-a Vehicle). Composition is generally preferred because it's more flexible (behaviour can be swapped at runtime) and avoids the tight coupling and fragile-base-class problems that deep inheritance hierarchies create - this is the idea behind 'favor composition over inheritance'.

Q5. What is object cloning, and what is the difference between a shallow copy and a deep copy?

Answer:

Object cloning creates a copy of an existing object. A shallow copy duplicates the object's top-level fields but nested objects/references are shared with the original (mutating a nested object affects both copies). A deep copy recursively duplicates nested objects too, so the copy is fully independent of the original.

Q6. Explain the difference between 'is-a' and 'has-a' relationships with examples.

Answer:

An 'is-a' relationship implies inheritance - a Dog is-a Animal. A 'has-a' relationship implies composition - a Car has-a Engine. Choosing 'has-a' over 'is-a' when there isn't a true specialization relationship avoids incorrectly forcing an inheritance hierarchy where composition would be more appropriate.

Q7. Explain the SOLID principles of object-oriented design.

Answer:

SOLID stands for: Single Responsibility (a class should have one reason to change), Open/Closed (open for extension, closed for modification), Liskov Substitution (subtypes must be substitutable for their base types without breaking correctness), Interface Segregation (prefer many small specific interfaces over one large one), and Dependency Inversion (depend on abstractions, not concrete implementations).

Q8. What are access modifiers? Explain public, private, protected, and default with examples.

Answer:

Access modifiers control visibility: 'public' - accessible from anywhere; 'private' - accessible only within the same class; 'protected' - accessible within the same package and by subclasses (even in different packages, in Java); 'default' (no modifier) - accessible only within the same package. They enforce encapsulation by controlling what other code can touch.

Q9. What is the Single Responsibility Principle, and why does violating it make code harder to maintain?

Answer:

The Single Responsibility Principle states a class should have only one reason to change - i.e., it should do one thing well. Violating it (e.g. a class that both parses files and sends emails) means unrelated changes end up modifying the same class, increasing the risk of breaking unrelated functionality and making the class harder to test and reuse.

DBMS & SQL

Q10. Explain the difference between a foreign key constraint and a check constraint.

Answer:

A foreign key constraint enforces referential integrity - it ensures a column's value matches an existing value in the referenced table's primary/unique key (or is NULL), preventing orphaned references. A check constraint enforces a custom condition on the values allowed in a column (e.g. age >= 18), independent of any other table.

Q11. Write a SQL query to find the Nth highest salary using a window function like RANK() or DENSE_RANK().

Answer:

SELECT name, salary FROM (SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM Employee) ranked WHERE rnk = N (substituting the desired N). DENSE_RANK() assigns ranks without gaps even when there are salary ties, making it more robust than ROW_NUMBER() for this use case.

Q12. Explain referential integrity and what happens when you try to delete a row referenced by a foreign key.

Answer:

Referential integrity ensures a foreign key value always points to an existing row in the referenced table (or is NULL). Trying to delete a row that is still referenced by a foreign key will fail with a constraint violation error, unless the foreign key was defined with ON DELETE CASCADE (deletes dependent rows too) or ON DELETE SET NULL (nulls out the reference) behavior.

Q13. How does a B-Tree index improve query performance internally?

Answer:

A B-Tree index keeps keys sorted in a balanced tree structure with high fan-out, so the database can locate a row's location in only a handful of disk-page reads (typically O(log n)) instead of scanning every row, by repeatedly narrowing the search range as it walks down the tree from the root to a leaf pointing at the actual row.

Q14. Explain the difference between horizontal and vertical partitioning of a table.

Answer:

Horizontal partitioning splits a table by rows (e.g. orders from 2023 in one partition, 2024 in another), similar in spirit to sharding but often within a single database instance. Vertical partitioning splits a table by columns (e.g. splitting rarely used large text/blob columns into a separate table), reducing the size of rows scanned for common queries that don't need those columns.

Q15. What is database replication, and what is the difference between synchronous and asynchronous replication?

Answer:

Database replication maintains copies of data across multiple servers for redundancy and read scaling. Synchronous replication waits for the replica to confirm the write before the transaction commits (stronger consistency, higher write latency); asynchronous replication commits on the primary immediately and propagates changes to replicas afterward (lower latency, but a small risk of data loss if the primary fails before replicating).

Q16. What are ACID vs BASE properties, and where would you prefer a BASE-compliant (NoSQL) system?

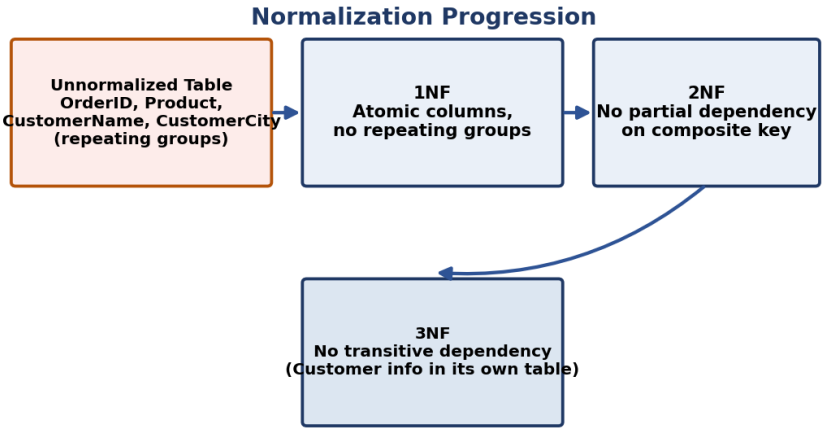
Answer:

ACID (Atomicity, Consistency, Isolation, Durability) prioritizes strict correctness and is the model used by traditional relational databases. BASE (Basically Available, Soft state, Eventual consistency) relaxes strict consistency in favor of availability and partition tolerance, common in NoSQL systems. You'd prefer a BASE system for massive-scale, highly available workloads (e.g. a social media feed) where brief inconsistency is acceptable in exchange for uptime and horizontal scalability.

Q17. What is normalization? Explain 1NF, 2NF, 3NF, and BCNF with a practical example.

Answer:

Normalization removes redundancy step by step: **1NF** requires atomic column values with no repeating groups; **2NF** requires 1NF plus no partial dependency of a non-key column on part of a composite primary key; **3NF** requires 2NF plus no transitive dependency (a non-key column depending on another non-key column); **BCNF** is a stricter version of 3NF handling certain anomalies where a table has multiple overlapping candidate keys. Example: splitting an Orders table that repeats CustomerCity for every order into a separate Customers table removes the transitive dependency.



Operating Systems

Q18. What is a context switch, and why does it have a performance cost?

Answer:

A context switch is the process of saving the CPU state (registers, program counter) of a currently running process/thread and loading the saved state of another, so execution can resume later exactly where it left off. It has a performance cost because the CPU does no useful work during the switch, and it can also cause cache/TLB misses afterward as the new process's data isn't cached yet.

Q19. What is the difference between internal and external fragmentation?

Answer:

Internal fragmentation is wasted space inside an allocated fixed-size block that's larger than what the process actually needs (common in paging). External fragmentation is wasted space between allocated blocks - free memory exists in total but is scattered in pieces too small individually to satisfy a new request (common in segmentation/variable-size allocation).

Q20. What is a critical section, and what conditions must a solution to the critical-section problem satisfy?

Answer:

A critical section is a segment of code that accesses shared resources and must not be executed by more than one process/thread concurrently to avoid race conditions. A valid solution must guarantee: Mutual Exclusion (only one process in the critical section at a time), Progress (a process not in the critical section can't block others from entering), and Bounded Waiting (a limit on how many times other processes can enter before a waiting process gets its turn).

Q21. Explain how a page fault is handled by the operating system.

Answer:

When a process accesses a page not currently in physical memory, the CPU raises a page fault trap. The OS then: locates the required page on disk, finds a free frame (or selects a victim page to evict via a page replacement algorithm like LRU, writing it back to disk if modified), loads the required page into that frame, updates the page table, and resumes the instruction that caused the fault.

Q22. Compare FCFS, SJF, Round Robin, and Priority scheduling algorithms - what are the trade-offs of each?

Answer:

FCFS (First-Come-First-Served) is simple but can cause the 'convoy effect' where a long job delays all others. SJF (Shortest Job First) minimizes average waiting time but requires knowing job lengths in advance and can starve long jobs. Round Robin gives each process a fixed time quantum, ensuring fairness and responsiveness for interactive systems, but too small a quantum increases context-switch overhead. Priority Scheduling runs the highest-priority job first but can starve low-priority jobs unless aging is used.

Q23. Explain the concept of multithreading and its benefits and challenges.

Answer:

Multithreading lets multiple threads within one process run concurrently, sharing memory and resources. Benefits include better CPU utilization on multi-core systems, faster context switching between threads than processes, and easier data sharing. Challenges include race conditions, deadlocks, and the added complexity of correctly synchronizing access to shared data.

Q24. What is a semaphore? Differentiate between binary and counting semaphores.

Answer:

A semaphore is an integer variable used for process synchronization, with atomic increment (signal/V) and decrement (wait/P) operations. A binary semaphore takes only values 0 or 1 and behaves like a mutex, ensuring exclusive access. A counting semaphore can take any non-negative integer, used to manage a pool of multiple identical resources (e.g. a fixed number of database connections).

Q25. Explain the difference between a monolithic kernel and a microkernel.

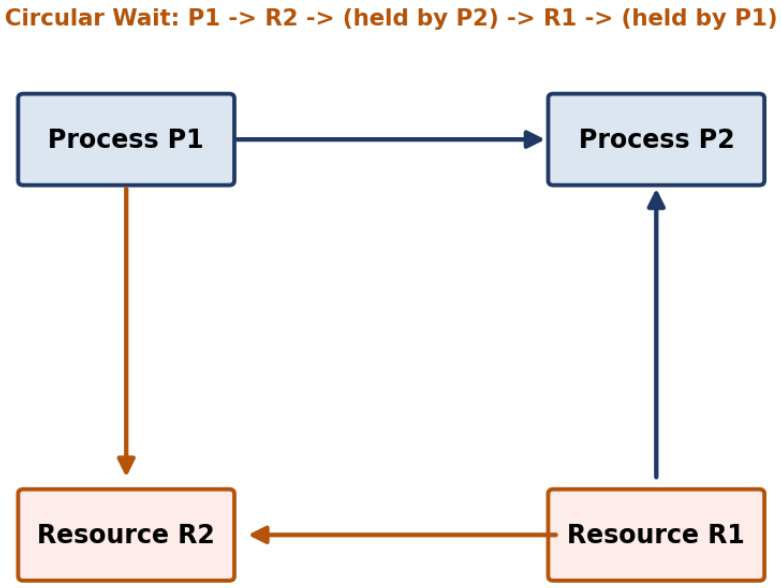
Answer:

A monolithic kernel runs all core OS services (file system, device drivers, memory management, scheduling) in a single address space in kernel mode, which is fast (no IPC overhead between components) but a bug in any component can crash the whole system. A microkernel keeps only the bare minimum (IPC, basic scheduling, memory management) in kernel space and moves other services to user-space processes, improving fault isolation and modularity at the cost of more inter-process communication overhead.

Q26. What is a deadlock, and describe three general strategies for handling deadlocks (prevention, avoidance, detection & recovery).

Answer:

A deadlock is a state where a set of processes are permanently blocked, each waiting for a resource held by another in the set. Prevention negates one of the four necessary conditions upfront (e.g. requiring resources to be requested all at once). Avoidance checks each request dynamically for safety (e.g. Banker's Algorithm). Detection & Recovery lets deadlocks happen but periodically checks for cycles in the resource-allocation graph and recovers by killing/rolling back a process to break the cycle.



Q27. What is thrashing, and what causes it?

Answer:

Thrashing happens when a system is so overcommitted on memory that it spends most of its time swapping pages in and out of disk rather than executing actual process instructions, causing CPU utilization and throughput to collapse even though the system appears busy. It's typically caused by too many processes running with too little physical memory, so each one's working set doesn't fit in RAM.

Computer Networks

Q28. Describe how you would troubleshoot a situation where a web application is reachable via IP but not via domain name.

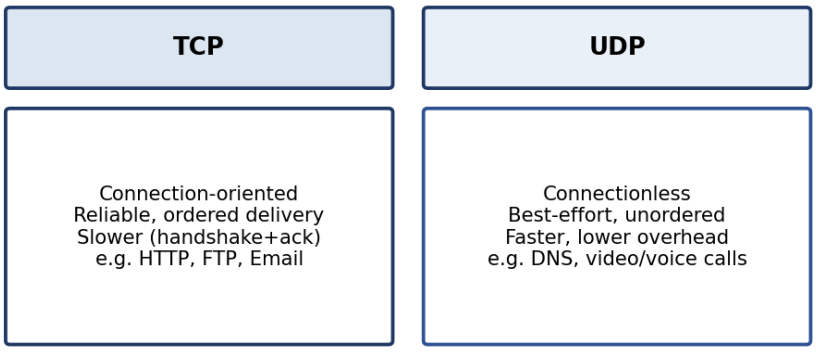
Answer:

Troubleshooting approach: first verify DNS resolution actually works (nslookup/dig the domain) since reachability by IP but not domain strongly suggests a DNS issue - check the domain's A/AAAA/CNAME records are correctly configured and propagated, check the local machine's DNS resolver/cache and hosts file, and confirm the domain registrar/DNS provider settings point to the correct IP or load balancer.

Q29. What is the difference between TCP and UDP? Give an example use case for each.

Answer:

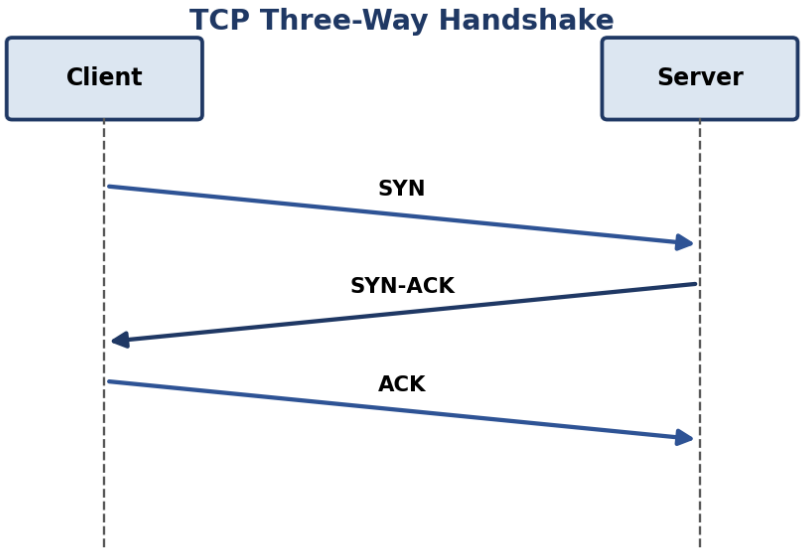
TCP is connection-oriented, establishes a session via a handshake, guarantees reliable and ordered delivery through acknowledgments and retransmission, but has more overhead - used for web pages, file transfers, and email. UDP is connectionless, sends packets with no delivery guarantee or ordering, but is faster with lower overhead - used for DNS lookups, video calls, and online gaming where speed matters more than occasional lost packets.



Q30. Explain the TCP three-way handshake process for establishing a connection.

Answer:

The three-way handshake establishes a reliable TCP connection: the client sends a SYN (synchronize) packet to the server; the server responds with a SYN-ACK, acknowledging the client's SYN and sending its own; the client responds with an ACK, acknowledging the server's SYN. After this, both sides have confirmed each other's readiness to send/receive, and data transfer begins.



Q31. Explain the seven layers of the OSI model and the role of each layer.

Answer:

The seven OSI layers, bottom to top: **Physical** - raw bit transmission over a medium; **Data Link** - framing and MAC addressing between directly connected nodes; **Network** - logical addressing and routing between networks (IP); **Transport** - end-to-end reliable/unreliable delivery (TCP/UDP); **Session** - managing sessions/connections between applications; **Presentation** - data formatting, encryption, compression; **Application** - the actual protocols apps use (HTTP, FTP, DNS).

Data Structures & Algorithms

Q32. Explain the two-pointer technique with an example problem.

Answer:

The two-pointer technique uses two indices moving through a data structure (often from opposite ends or at different speeds) to avoid nested loops. Example: given a sorted array, to find a pair summing to a target, start one pointer at the beginning and one at the end - if the sum is too large, move the right pointer left; if too small, move the left pointer right - solving the problem in O(n) instead of the O(n^2) brute force of checking every pair.

Q33. Explain the difference between time complexity and space complexity with examples.

Answer:

Time complexity measures how an algorithm's runtime grows as input size increases (e.g. O(n) for a single loop through n elements). Space complexity measures how much additional memory an algorithm needs relative to input size (e.g. O(1) for an in-place swap vs O(n) for creating a copy of the array). Both are usually expressed using Big-O notation and both matter when choosing an algorithm for constrained environments.

Q34. What is Big-O notation, and why do we use asymptotic analysis instead of exact runtimes?

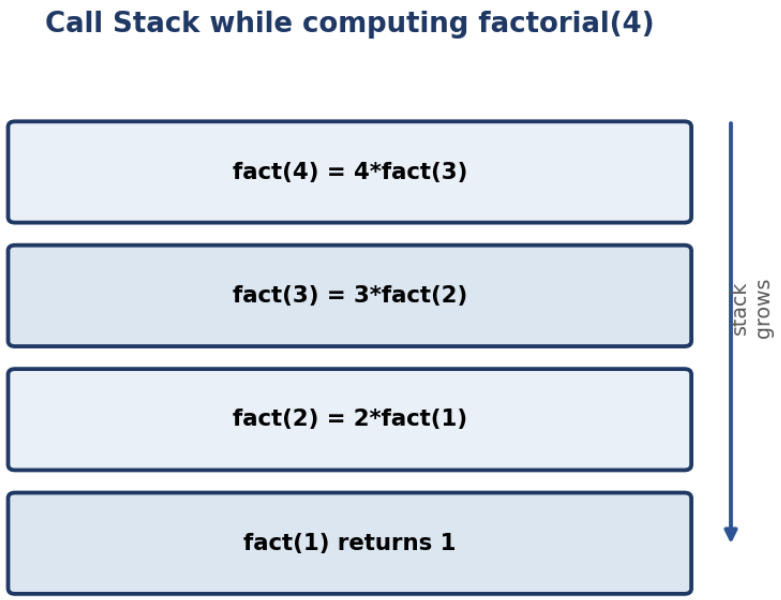
Answer:

Big-O notation describes the upper-bound growth rate of an algorithm's running time or space as input size approaches infinity, ignoring constant factors and lower-order terms. We use asymptotic analysis instead of exact runtimes because exact timings vary by hardware, language, and implementation details, while Big-O gives a hardware-independent way to compare how algorithms scale as input grows large.

Q35. What is dynamic programming, and how does it differ from plain recursion and from greedy algorithms?

Answer:

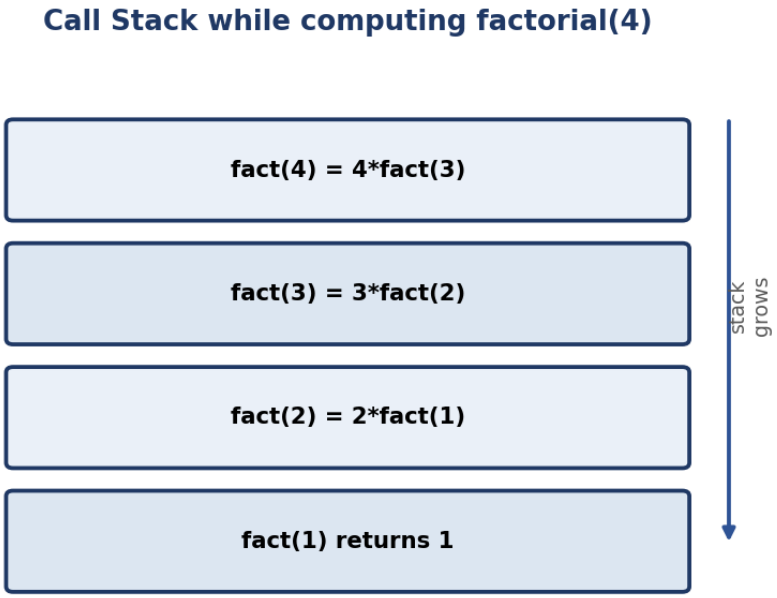
Dynamic programming solves problems by breaking them into overlapping subproblems, solving each just once, and storing (memoizing) the results to avoid redundant recomputation - it's applicable when a problem has both optimal substructure and overlapping subproblems. Plain recursion without memoization re-solves the same subproblems repeatedly, often leading to exponential time. Unlike a greedy algorithm (which makes one irrevocable locally-optimal choice at each step), DP considers combinations of subproblem solutions to guarantee a globally optimal result when applicable.



Q36. Explain how recursion uses the call stack internally, and what causes a stack overflow.

Answer:

Each recursive call pushes a new stack frame onto the call stack, containing that call's local variables, parameters, and return address. As calls nest deeper, more frames stack up; when a call returns, its frame is popped off. If recursion goes too deep (e.g. no base case, or a base case never reached due to a bug), the call stack exceeds its allocated size, causing a stack overflow.



Q37. Explain amortized time complexity using the example of a dynamic array's append operation.

Answer:

Amortized time complexity averages the cost of an operation over a sequence of operations, even if occasional individual operations are expensive. A dynamic array's append is usually O(1), but when the underlying fixed-size array is full, it must allocate a new (typically double-sized) array and copy all existing elements - an O(n) operation. Because this expensive resize happens increasingly rarely as the array grows, the average cost per append across many operations still works out to O(1) amortized.

Q38. How would you find the kth largest element in an unsorted array efficiently?

Answer:

One efficient approach uses a min-heap of size k: iterate through the array, pushing elements onto the heap and popping the smallest whenever the heap exceeds size k - after processing all elements, the heap's root is the kth largest, achieving O(n log k) time. Alternatively, Quickselect (a partition-based approach related to Quicksort) finds it in average O(n) time.

Q39. Explain how a trie data structure works and where it is useful (e.g., autocomplete).

Answer:

A trie (prefix tree) stores strings character by character along tree edges, where each path from the root represents a prefix, and nodes are often marked to indicate a complete word ends there. It's especially useful for prefix-based operations like autocomplete/typeahead suggestions, spell-checking, and IP routing tables, since searching or inserting a string takes time proportional to the string's length rather than the number of stored strings.

Q40. What is backtracking, and how is it different from brute force?

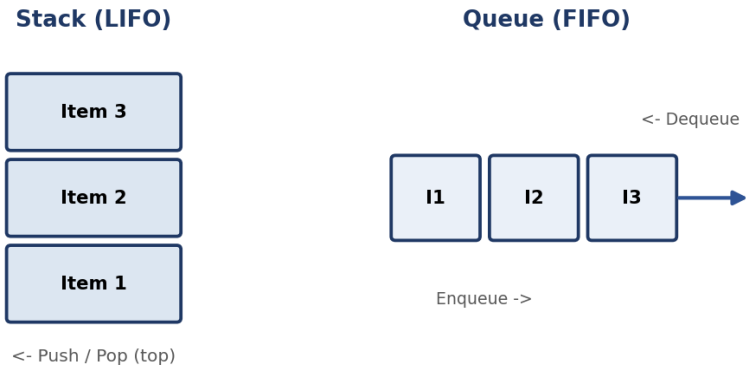
Answer:

Backtracking systematically explores all candidate solutions, but abandons ('backtracks' from) a partial candidate as soon as it determines that candidate cannot possibly lead to a valid solution, rather than exploring it to completion. This pruning makes it far more efficient than pure brute force, which would fully explore every possibility regardless of early signs of failure - classic examples include the N-Queens problem and Sudoku solvers.

Q41. What is the difference between a stack and a queue, and where is each used in real systems?

Answer:

A stack follows Last-In-First-Out (LIFO) order - used for undo functionality, expression evaluation, and the call stack behind recursion. A queue follows First-In-First-Out (FIFO) order - used for task scheduling, breadth-first search, and handling requests in the order they arrive (e.g. print queues, message queues).



Q42. What is memoization, and how does it improve the performance of recursive algorithms?

Answer:

Memoization stores the result of a function call keyed by its input parameters (often in a hashmap or array), so if the same inputs occur again, the cached result is returned instantly instead of recomputing it. This turns algorithms with exponential-time naive recursion (like plain recursive Fibonacci) into polynomial-time algorithms by eliminating redundant recursive calls on the same subproblem.

Q43. How would you merge two sorted arrays/linked lists efficiently?

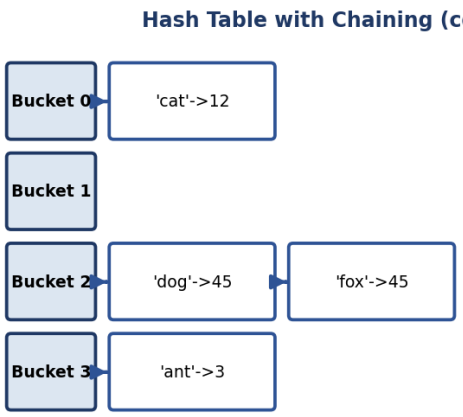
Answer:

Use a two-pointer merge: maintain one pointer into each sorted array/list, repeatedly compare the elements they point to, append the smaller one to the result, and advance that pointer - continue until one input is exhausted, then append the remainder of the other. This runs in O(n + m) time, which is also the core merge step used inside Merge Sort.

Q44. Explain how a hash table works internally, including collision resolution strategies.

Answer:

A hash table maps keys to array indices using a hash function, giving average O(1) time for insert/search/delete. Since different keys can hash to the same index (a collision), collisions are typically resolved via chaining (each bucket holds a linked list/array of entries that hashed there) or open addressing (probing for the next free slot in the array itself, e.g. linear or quadratic probing).



Q45. Explain how binary search works and derive its time complexity.

Answer:

Binary search repeatedly divides a sorted array in half: compare the target to the middle element, and discard the half that can't contain the target, narrowing the search range each time. Since the search space is halved every step, the time complexity is O(log n) - a huge improvement over linear search's O(n) for large sorted datasets.

Q46. Explain Dijkstra's algorithm for shortest paths and its limitations with negative edge weights.

Answer:

Dijkstra's algorithm finds the shortest path from a source node to all others in a weighted graph by repeatedly selecting the unvisited node with the smallest known distance, then 'relaxing' (updating) the distances of its neighbors - using a priority queue/min-heap, it runs in O(E log V). Its key limitation is that it assumes all edge weights are non-negative; a negative edge can invalidate its greedy assumption that once a node is finalized its distance can't improve, requiring an algorithm like Bellman-Ford instead.

Q47. Explain topological sorting and where it is used.

Answer:

Topological sorting produces a linear ordering of a Directed Acyclic Graph's (DAG) vertices such that for every directed edge u -> v, u comes before v in the ordering. It's used wherever tasks have dependencies that must be respected, such as course prerequisite ordering, build systems compiling files in the right order, or task scheduling pipelines.

Q48. What is the difference between a greedy algorithm and dynamic programming? Give an example problem for each.

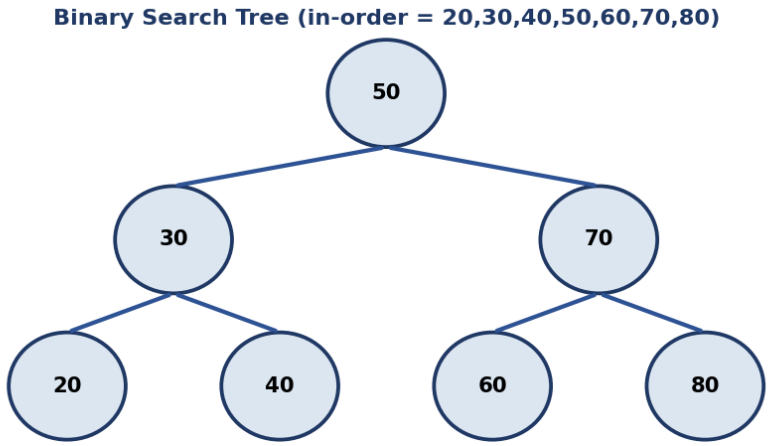
Answer:

A greedy algorithm makes the locally optimal choice at each step without reconsidering past decisions, and works only for problems where local optimality leads to a global optimum (e.g. activity/interval selection, Huffman coding). Dynamic programming considers overlapping subproblems and combines their optimal solutions, guaranteeing a global optimum for a broader class of problems (e.g. the 0/1 Knapsack problem, where greedy choice-by-choice doesn't always give the best result).

Q49. What is the difference between a balanced and an unbalanced binary search tree, and why does balance matter?

Answer:

A balanced BST keeps its height close to log n by rebalancing after insertions/deletions (e.g. AVL trees, Red-Black trees), guaranteeing O(log n) search/insert/delete. An unbalanced BST can degenerate into something resembling a linked list if elements are inserted in sorted order without rebalancing, degrading all operations to O(n). Balance matters because it's what actually delivers the logarithmic performance a BST is chosen for in the first place.



Q50. Compare Merge Sort and Quick Sort - discuss best/worst case complexity, stability, and when you'd choose one over the other.

Answer:

Merge Sort has guaranteed O(n log n) time in best, average, and worst cases, is stable, but requires O(n) extra space for merging. Quick Sort also averages O(n log n) but can degrade to O(n^2) in the worst case (e.g. already-sorted input with a poor pivot choice), is typically not stable, but sorts in-place with better real-world constant factors and cache performance. Choose Merge Sort when stability or guaranteed worst-case performance matters (e.g. external sorting); choose Quick Sort for general in-memory sorting where average-case speed matters more.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No

Q51. What is the sliding window technique, and in what type of problems is it useful?

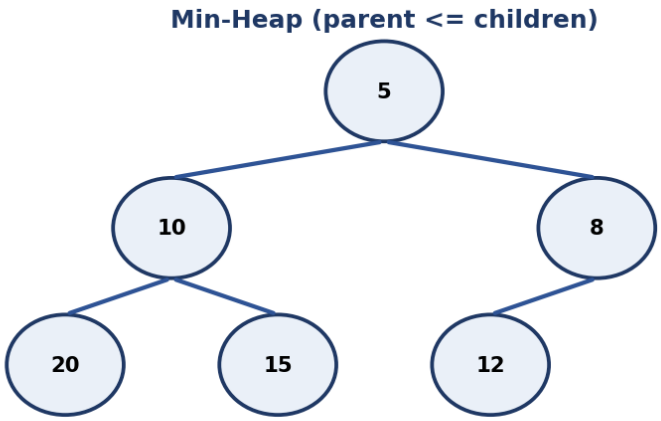
Answer:

The sliding window technique maintains a contiguous subrange ('window') over an array or string and expands/shrinks its boundaries incrementally rather than recomputing from scratch for every possible subrange. It's especially useful for problems asking for the best/longest/shortest subarray or substring satisfying some condition (e.g. maximum sum subarray of size k, or longest substring without repeating characters), typically turning an O(n^2) brute force into an O(n) solution.

Q52. Explain how a min-heap/max-heap is implemented and how it supports efficient priority queue operations.

Answer:

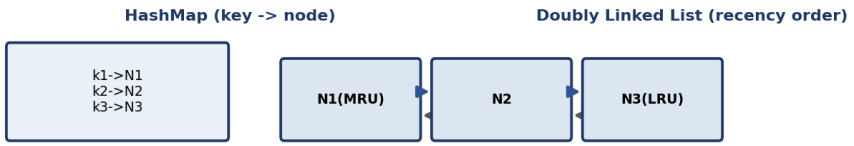
A heap is a complete binary tree (usually implemented as an array) satisfying the heap property: in a min-heap, every parent is less than or equal to its children (so the minimum is always at the root); in a max-heap, every parent is greater than or equal to its children. Insertion and extraction of the min/max both take O(log n) since you only need to 'bubble' an element up or down the height of the tree, making heaps the standard backing structure for an efficient priority queue.



Q53. Explain how you would design an LRU cache and the data structures you'd use to achieve O(1) get/put operations.

Answer:

An LRU cache needs O(1) get and put operations. The standard design combines a hashmap (key -> pointer to a node) for instant lookup with a doubly linked list ordered by recency of use: on access, the accessed node is unlinked and moved to the 'most recently used' end in O(1); when the cache exceeds capacity, the node at the 'least recently used' end is evicted, also in O(1), with the hashmap updated accordingly.



O(1) get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Coding Problems

Q54. Write a program to implement a queue using a fixed-size array (circular queue).

Answer:

A circular queue uses a fixed-size array with 'front' and 'rear' indices that wrap around using modulo arithmetic (rear = (rear + 1) % capacity) once they reach the end of the array, allowing the queue to reuse freed space at the beginning after elements have been dequeued - avoiding the wasted space of a naive fixed-array queue that never reuses earlier slots.

Q55. Write a program to implement binary search on a rotated sorted array.

Answer:

Modify binary search by first determining which half of the array (from mid to low, or mid to high) is properly sorted, then checking if the target lies within that sorted half's range - if so, search recurse/iterate into that half, otherwise search the other half. This preserves the O(log n) time complexity of standard binary search despite the rotation.

Q56. Write a program to check whether a given string is a palindrome.

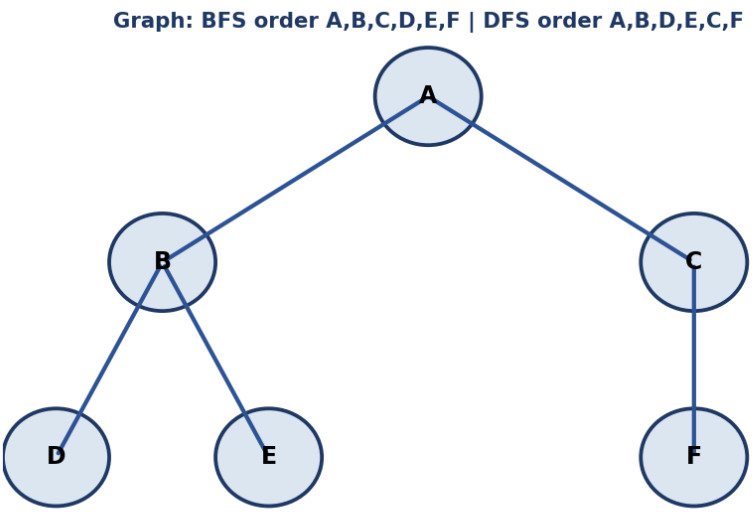
Answer:

Use two pointers, one from the start (left) and one from the end (right) of the string, comparing characters and moving both pointers inward - if any pair doesn't match, it's not a palindrome; if the pointers meet or cross without mismatches, it is. This runs in O(n) time and O(1) extra space (ignoring the input itself).

Q57. Write a program to count the number of islands in a 2D grid (graph/matrix traversal).

Answer:

Treat the grid as an implicit graph where each land cell ('1') is a node connected to its up/down/left/right land neighbors. Scan every cell; whenever an unvisited land cell is found, increment the island count and run a BFS or DFS from that cell to mark every connected land cell as visited, so it isn't counted again. This runs in O(rows * cols) time since each cell is visited a constant number of times.



Q58. Write a program to find all pairs in an array that sum up to a given target value.

Answer:

Use a hashmap: iterate through the array once, and for each element x, check if (target - x) already exists in the hashmap - if so, record the pair; otherwise, add x to the hashmap and continue. This finds all pairs summing to the target in O(n) time and O(n) space, instead of the O(n^2) brute-force nested loop.

Q59. Write a program to flatten a nested list/array.

Answer:

Use recursion: iterate through each element of the (possibly nested) list - if an element is itself a list, recursively flatten it and extend the result with its contents; if it's a plain value, append it directly to the result list. This naturally handles arbitrary nesting depth, with time complexity O(n) where n is the total number of elements across all nesting levels.

Q60. Write a program to determine if a given number is prime, and optimize it for large inputs.

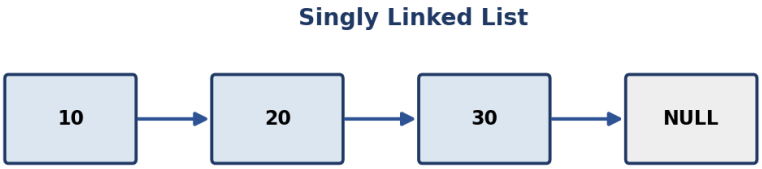
Answer:

Check divisibility only up to the square root of the number (if n has a factor larger than sqrt(n), it must also have a corresponding factor smaller than sqrt(n)), skipping even numbers after checking 2, which reduces the check to roughly O(sqrt(n)). For checking primality of many numbers up to a limit, the Sieve of Eratosthenes precomputes all primes up to that limit in O(n log log n), which is far more efficient than testing each number individually.

Q61. Write a program to reverse a singly linked list (iterative and recursive approaches).

Answer:

Iterative: maintain a 'previous' pointer (initially null) and a 'current' pointer (initially head); at each step, save current.next, point current.next to previous, then advance previous and current forward - repeat until current is null, then previous is the new head. Recursive: recurse to the end of the list first, then reverse the pointer of each node on the way back up (node.next.next = node; node.next = null). Both run in O(n) time; iterative uses O(1) extra space while recursive uses O(n) call-stack space.



Q62. Write a program to check if two strings are anagrams of each other.

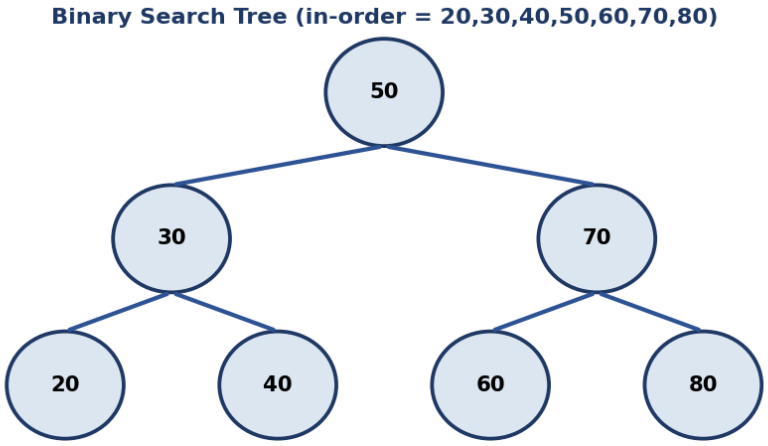
Answer:

Two common approaches: (1) sort both strings and check if the sorted versions are equal (O(n log n)); or (2) build a frequency count (e.g. a 26-length array for lowercase letters, or a hashmap for general characters) of one string, then decrement counts while scanning the second string - if all counts return to zero and lengths match, they're anagrams (O(n) time, O(1) or O(k) space).

Q63. Write a program to find the lowest common ancestor (LCA) of two nodes in a binary tree.

Answer:

For a BST, the LCA can be found by starting at the root and comparing both target node values to the current node's value: if both are smaller, move to the left child; if both are larger, move to the right child; if they diverge (one smaller, one larger, or one equals the current node), the current node is the LCA. This runs in O(h) time where h is the tree height. For a general binary tree (not necessarily a BST), a recursive post-order search that returns the first node where both targets are found in different subtrees is used instead.



Q64. Write a program to merge overlapping intervals given a list of intervals.

Answer:

Sort the intervals by their start time, then iterate through them while maintaining a 'current merged interval' - if the next interval's start is less than or equal to the current merged interval's end, merge them by extending the end to the maximum of the two ends; otherwise, push the current merged interval to the result and start a new one with the next interval. This runs in O(n log n) time due to the initial sort.

Q65. Write a program to find the longest common subsequence between two strings.

Answer:

Use dynamic programming: build a 2D table dp[i][j] representing the length of the longest common subsequence between the first i characters of string A and first j characters of string B. If A[i-1] == B[j-1], dp[i][j] = dp[i-1][j-1] + 1; otherwise dp[i][j] = max(dp[i-1][j], dp[i][j-1]). This runs in O(n*m) time and space, where n and m are the string lengths.

Q66. Write a program to find the maximum subarray sum (Kadane's Algorithm).

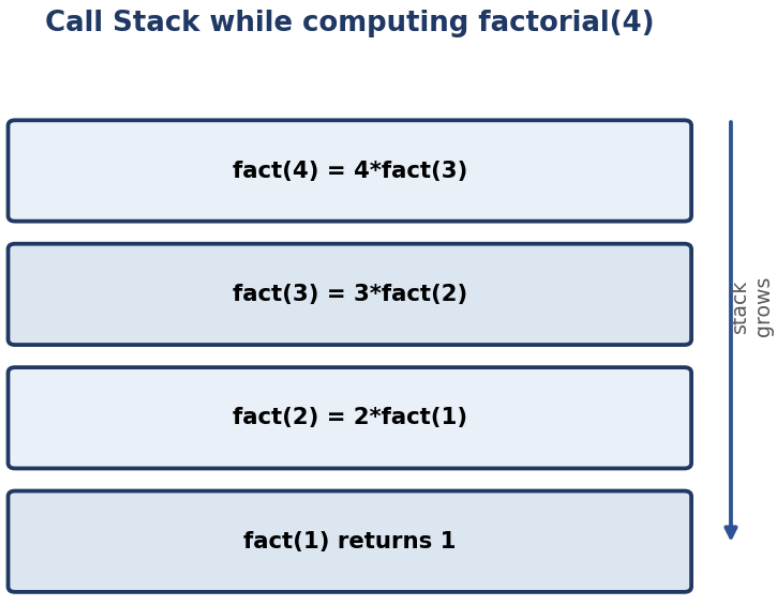
Answer:

Kadane's Algorithm scans the array once, maintaining a running 'current sum' that resets to the current element whenever adding the previous running sum would make it smaller than the element alone (i.e. current_sum = max(arr[i], current_sum + arr[i])), while tracking the overall maximum seen so far. This finds the maximum subarray sum in O(n) time and O(1) space.

Q67. Write a program to find the factorial of a number using recursion.

Answer:

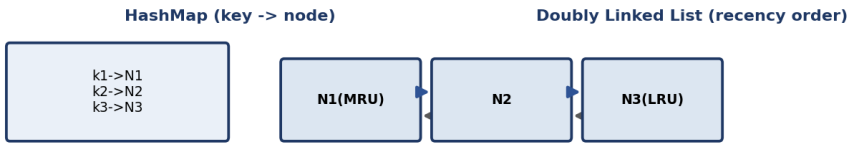
Recursive definition: factorial(n) = n * factorial(n-1), with a base case factorial(0) = 1 (or factorial(1) = 1). Each call multiplies n by the result of the smaller subproblem until the base case is reached, then the multiplications unwind back up the call stack. Time complexity is O(n); space complexity is O(n) due to the recursive call stack.



Q68. Write a program to implement a basic LRU cache using a hashmap and a doubly linked list.

Answer:

Maintain a hashmap from key to a reference to its node in a doubly linked list ordered by recency of use. On get(key): if present, move that node to the front (most recently used end) and return its value, else return -1/None. On put(key, value): if the key exists, update its value and move it to the front; otherwise insert a new node at the front, and if capacity is exceeded, remove the node at the back of the list (least recently used) along with its hashmap entry. Both operations run in O(1).

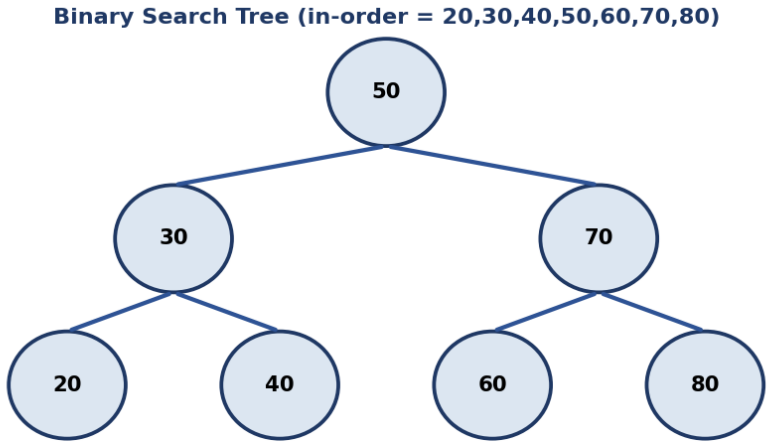


O(1) get/put: hashmap finds the node instantly, doubly linked list reorders it to MRU end

Q69. Write a program to check if a given binary tree is a valid Binary Search Tree.

Answer:

Perform an in-order traversal of the tree and check that the resulting sequence of values is strictly increasing - a valid BST always produces a sorted sequence under in-order traversal. Alternatively, recursively validate each node against a running (min, max) valid range inherited from its ancestors, narrowing the range as you descend left or right - both approaches run in O(n) time.



Q70. Write a program to find the missing number in an array containing n distinct numbers from 0 to n.

Answer:

Sum formula approach: the expected sum of numbers 0 to n is n*(n+1)/2; subtract the actual sum of the given array from this expected sum, and the difference is the missing number - O(n) time, O(1) space. Alternatively, XOR every number from 0 to n together with every element in the array; since XOR-ing a number with itself cancels out to zero, all present numbers cancel, leaving only the missing number.

Q71. Write a program to check whether a binary tree is balanced.

Answer:

Recursively compute the height of the left and right subtrees for every node; if at any point the difference in height between left and right subtrees exceeds 1 for any node, the tree is unbalanced. An efficient implementation computes height and checks balance in the same bottom-up traversal (returning -1 as a sentinel to signal 'unbalanced' early), achieving O(n) time instead of the naive O(n^2).

Q72. Write a program to rotate an array by k positions.

Answer:

Reverse the whole array, then reverse the first k elements, then reverse the remaining n-k elements - this three-step reversal rotates the array by k positions in-place in O(n) time and O(1) extra space. (Alternatively, use an auxiliary array of size n to place each element directly at its rotated index, trading O(n) space for simpler code.)

Q73. Write a program to implement quicksort or mergesort from scratch.

Answer:

Quicksort: pick a pivot element, partition the array so elements smaller than the pivot come before it and larger elements come after, then recursively sort the two partitions - average O(n log n), worst case O(n^2) with a poor pivot choice. Mergesort: recursively split the array in half until subarrays of size 1 remain, then repeatedly merge sorted subarrays back together - guaranteed O(n log n) in all cases, but needs O(n) auxiliary space for merging.

Sorting Algorithm Complexity Comparison

Algorithm	Best	Average	Worst	Stable?
Bubble Sort	O(n)	O(n^2)	O(n^2)	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	No
Insertion Sort	O(n)	O(n^2)	O(n^2)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n^2)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	No

Q74. Write a program to find the middle element of a linked list in a single pass.

Answer:

Use the 'slow and fast pointer' technique: the slow pointer advances one node at a time while the fast pointer advances two nodes at a time - when the fast pointer reaches the end of the list, the slow pointer will be exactly at the middle. This finds the middle node in a single pass, in O(n) time and O(1) space.

Q75. Write a program to remove duplicates from a sorted array in-place.

Answer:

Since the array is already sorted, use a slow pointer marking the last position of a unique element written so far, and a fast pointer scanning ahead - whenever the fast pointer finds a value different from the one at the slow pointer, advance the slow pointer and copy that new value into place. This removes duplicates in-place in O(n) time and O(1) extra space.

Q76. Write a program to find the first non-repeating character in a string.

Answer:

Use a hashmap to count the frequency of each character in a single pass through the string, then iterate through the string again in order and return the first character whose count is exactly 1. This runs in O(n) time and O(k) space, where k is the size of the character set.

Q77. Write a program to find the intersection of two arrays.

Answer:

Efficient approach: put all elements of the smaller array into a hash set for O(1) lookup, then iterate through the other array and collect any element found in the set (adding it to a result set to avoid duplicates in the output). This runs in O(n + m) time and O(min(n, m)) space.

Programming Language Fundamentals

Q78. What is method chaining, and how would you implement it in a class you design?

Answer:

Method chaining lets you call multiple methods on the same object in a single statement, e.g. builder.setName('X').setAge(30).build(). To implement it, each method that would normally return void instead returns 'this' (the current object instance), allowing the next method call to be appended directly onto the previous one's result - commonly used in builder-pattern classes for constructing complex objects readably.

Q79. What is a memory leak, and how would you identify one in a long-running application?

Answer:

A memory leak occurs when a program allocates memory but never releases it even though it's no longer needed, causing memory usage to grow over time and potentially exhausting available memory. In a long-running application, you'd identify one using a memory profiler to take heap snapshots over time and look for object counts/memory usage that keep growing without bound, then trace those objects back to what's still holding references to them (e.g. an ever-growing cache or a forgotten event-listener registration).

Q80. Explain the difference between an abstract class and a concrete class with an example from a language you know well.

Answer:

An abstract class can declare abstract methods (no body, must be implemented by subclasses) alongside concrete methods with actual implementations, and cannot be instantiated directly - e.g. an abstract Shape class with an abstract area() method but a concrete describe() method common to all shapes. A concrete class provides full implementations for all its methods and can be instantiated directly, e.g. a Circle class extending Shape and implementing area().

Q81. Explain the difference between static and dynamic memory allocation.

Answer:

Static memory allocation reserves a fixed amount of memory at compile time (e.g. a fixed-size array or a local variable on the stack), which is fast but inflexible since the size must be known ahead of time. Dynamic memory allocation reserves memory at runtime (e.g. via malloc/new), allowing the program to request exactly the amount of memory it needs based on runtime conditions, at the cost of manual (or garbage-collected) management and slightly slower access.

Q82. What is exception handling, and why is it important to avoid empty catch blocks?

Answer:

Exception handling is the mechanism for detecting and responding to runtime errors (like a missing file or invalid input) gracefully, using constructs like try/catch/finally, instead of letting the program crash. Empty catch blocks are dangerous because they silently swallow errors - the program continues running as if nothing went wrong, making it far harder to diagnose bugs later since there's no log, message, or fallback behavior indicating that something actually failed.

Q83. What are lambda expressions/anonymous functions, and when would you use them?

Answer:

A lambda expression (or anonymous function) is a small, unnamed function defined inline, typically used for short operations passed as arguments to other functions - e.g. sorting a list with a custom key: sorted(items, key=lambda x: x.age). They're best used for simple, one-off logic; anything more complex or reused in multiple places is usually clearer as a named function.

Q84. Explain the concept of multithreading vs multiprocessing, and when you would choose one over the other.

Answer:

Multithreading runs multiple threads within a single process, sharing memory - lightweight and fast to communicate between threads, but requires careful synchronization to avoid race conditions, and (in languages like Python with a GIL) doesn't achieve true CPU parallelism. Multiprocessing runs separate processes, each with its own memory space - avoids shared-memory race conditions and achieves true parallelism across CPU cores, but has higher overhead for creation and inter-process communication. Choose multithreading for I/O-bound work and multiprocessing for CPU-bound work in GIL-constrained languages.

Web & Cloud Fundamentals

Q85. What is containerization (e.g., Docker), and how is it different from a virtual machine?

Answer:

Containerization packages an application with its dependencies and runtime into a lightweight, portable container (e.g. via Docker) that shares the host OS kernel, making containers fast to start and resource-efficient. A virtual machine, by contrast, virtualizes an entire hardware stack including its own full guest OS, making VMs heavier and slower to start but providing stronger isolation between workloads.

Q86. Explain the difference between authentication and authorization, and how OAuth 2.0 fits in.

Answer:

Authentication verifies who a user is (e.g. checking a username/password or token), while authorization determines what an authenticated user is allowed to do (e.g. which resources/actions they can access). OAuth 2.0 is an authorization framework that lets a user grant a third-party application limited access to their resources on another service (e.g. 'Sign in with Google') without sharing their actual password, typically issuing access tokens that represent specific granted permissions.

Q87. What is auto-scaling in cloud computing, and what metrics typically trigger it?

Answer:

Auto-scaling automatically adjusts the number of running compute instances up or down based on real-time demand, rather than requiring manual intervention. Common triggering metrics include CPU utilization, memory usage, request/queue length, and response latency - when a threshold is crossed for a sustained period, the auto-scaler adds or removes instances accordingly.

Q88. Explain the concept of statelessness in REST APIs and why it matters for scalability.

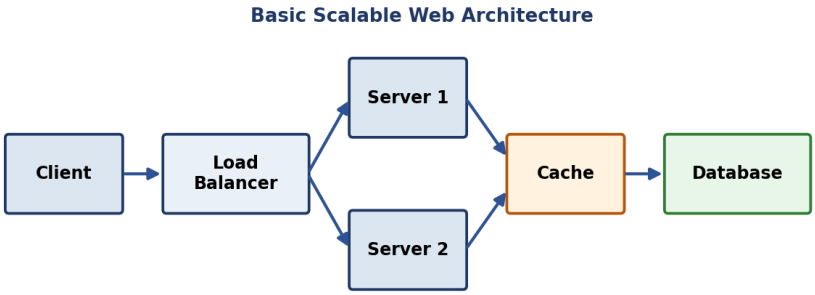
Answer:

Statelessness means each request from a client to a server must contain all the information the server needs to process it, with no reliance on server-side session state stored between requests. This matters for scalability because any server instance can handle any request without needing shared session state, making it trivial to add more server instances behind a load balancer without worrying about which server 'remembers' a given client.

Q89. What is the purpose of an API Gateway in a microservices-based application?

Answer:

An API Gateway sits in front of a microservices architecture as the single entry point for client requests, routing each request to the appropriate backend service. It commonly also handles cross-cutting concerns centrally - authentication/authorization, rate limiting, request/response transformation, and logging - so individual microservices don't each need to reimplement that logic.



Q90. What is a RESTful API, and what makes an API design 'RESTful'?

Answer:

A RESTful API is an API designed around REST (Representational State Transfer) architectural principles: it uses standard HTTP methods (GET, POST, PUT, DELETE) mapped to resource operations, is stateless (each request contains all information needed), uses resource-based URLs (e.g. /users/123), and typically returns representations of resources (often JSON). An API is 'RESTful' to the extent it actually follows these constraints rather than just using HTTP as a transport.

Q91. Explain how a CDN improves website performance for geographically distributed users.

Answer:

A CDN caches static content (images, CSS, JS, video) on edge servers distributed across many geographic locations. When a user requests that content, it's served from the nearest edge server instead of traveling all the way to the origin server, reducing round-trip latency, offloading traffic from the origin, and improving load times especially for users far from the origin's data center.

Q92. Explain how a message queue like Kafka or RabbitMQ helps decouple microservices.

Answer:

A message queue like Kafka or RabbitMQ sits between producer services (that generate events/messages) and consumer services (that process them), letting producers publish messages without waiting for consumers to be ready or fast. This decouples services in time and load - producers and consumers can scale, fail, or be deployed independently - and provides buffering so a burst of traffic doesn't overwhelm downstream consumers directly.

System Design Basics

Q93. Explain how you would design a notification system that can send millions of push notifications reliably.

Answer:

A reliable large-scale notification system typically uses a message queue (e.g. Kafka) to decouple the event that triggers a notification from the actual delivery: the triggering service publishes an event to the queue, and a pool of worker consumers pulls from the queue and calls the relevant push notification provider (e.g. FCM/APNs) for each recipient, retrying failed deliveries with backoff. This design absorbs traffic spikes, allows independent scaling of ingestion vs delivery workers, and avoids losing notifications if a downstream provider is temporarily slow or unavailable.

Q94. How would you design a rate limiter for an API?

Answer:

A common design uses the 'token bucket' or 'sliding window' algorithm: each client is allocated a bucket of tokens that refills at a fixed rate, and each request consumes a token - requests are rejected (or queued) once the bucket is empty, until it refills. This state (token counts and timestamps) is typically stored in a fast shared store like Redis so the rate limiter works consistently across multiple servers handling the same client's requests.

Q95. Explain the trade-offs between strong consistency and eventual consistency in a distributed system.

Answer:

Strong consistency guarantees that once a write completes, every subsequent read (from any replica) sees that write immediately - simpler to reason about, but often requires more coordination between nodes, increasing latency and reducing availability during network issues. Eventual consistency allows replicas to temporarily diverge after a write, guaranteeing only that all replicas will converge to the same value if no new writes occur for some time - offering better availability and lower latency, at the cost of applications occasionally reading stale data.

Q96. What is idempotency, and why is it important when designing payment or retry-safe APIs?

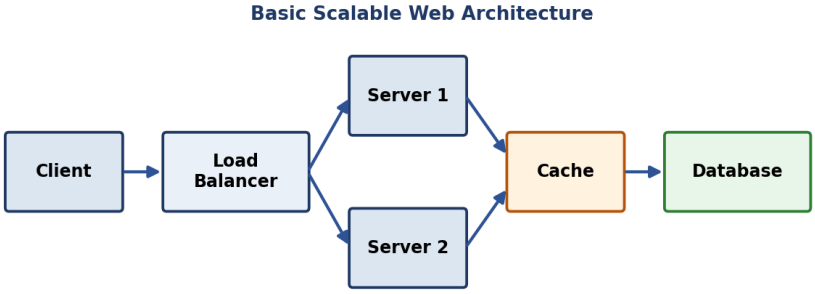
Answer:

Idempotency means performing an operation multiple times has the same effect as performing it once. It's crucial for payment or retry-safe APIs because network failures often make clients unsure whether a request actually succeeded, causing them to retry - without idempotency, a retried payment request could charge a customer twice. It's typically implemented by having the client generate a unique idempotency key per logical operation, which the server stores and checks against on each retry to avoid re-processing the same request.

Q97. What is the difference between horizontal and vertical scaling, and when would you use each?

Answer:

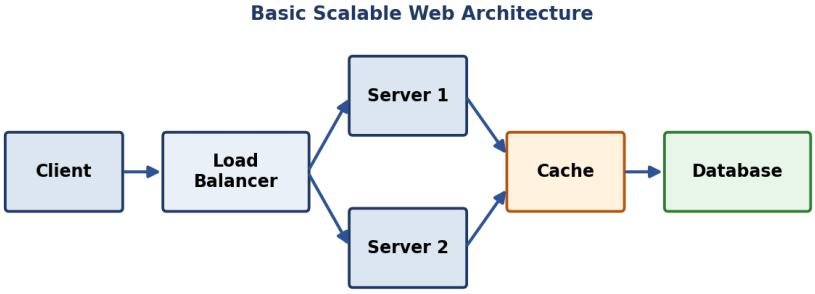
Horizontal scaling adds more machines/servers to distribute load across them (scale out), which is generally more resilient (no single point of failure) and has no hard upper limit, but requires the application to be designed to run across multiple nodes (e.g. stateless services, distributed data). Vertical scaling increases the resources (CPU/RAM) of a single existing machine (scale up), which is simpler with no architecture changes needed, but is limited by the maximum capacity of a single machine and doesn't improve fault tolerance.



Q98. What is a load balancer, and what are common load-balancing strategies (round robin, least connections, etc.)?

Answer:

A load balancer distributes incoming client requests across multiple backend servers to prevent any single server from being overwhelmed and to improve availability if a server goes down. Common strategies: Round Robin (cycles requests evenly across servers), Least Connections (routes to the server currently handling the fewest active connections), Weighted Round Robin (accounts for servers with different capacities), and IP Hash (routes based on client IP for session stickiness).



Q99. What is database sharding, and what challenges does it introduce (e.g., cross-shard queries)?

Answer:

Database sharding splits a large table's data horizontally across multiple database servers based on a shard key, so each shard holds only a subset of rows, improving write throughput and storage scalability beyond what one machine can handle. Challenges include queries or transactions that need data spanning multiple shards (requiring expensive cross-shard joins or scatter-gather queries), rebalancing data when adding/removing shards, and maintaining a consistent, well-distributed choice of shard key to avoid 'hot' shards.

Q100. How would you design a basic caching layer for a high-traffic read-heavy application?

Answer:

For a high-traffic, read-heavy application, place a cache (e.g. Redis or Memcached) between the application servers and the database. On a read, check the cache first (a 'cache hit' returns data instantly without touching the database); on a miss, read from the database and populate the cache for next time. Choose a cache invalidation/expiration strategy (TTL, or explicit invalidation on writes) to balance freshness against cache-hit rate, and consider a cache-aside or write-through pattern depending on how tolerant the application is of slightly stale reads.

Note: 23 of these 100 questions include a supporting diagram. Content compiled from publicly available 2026 placement-prep resources and established CS-fundamentals question banks - verify current-year specifics before your interview. Practice explaining answers out loud or in writing, not just reading - recall under pressure is the real test.